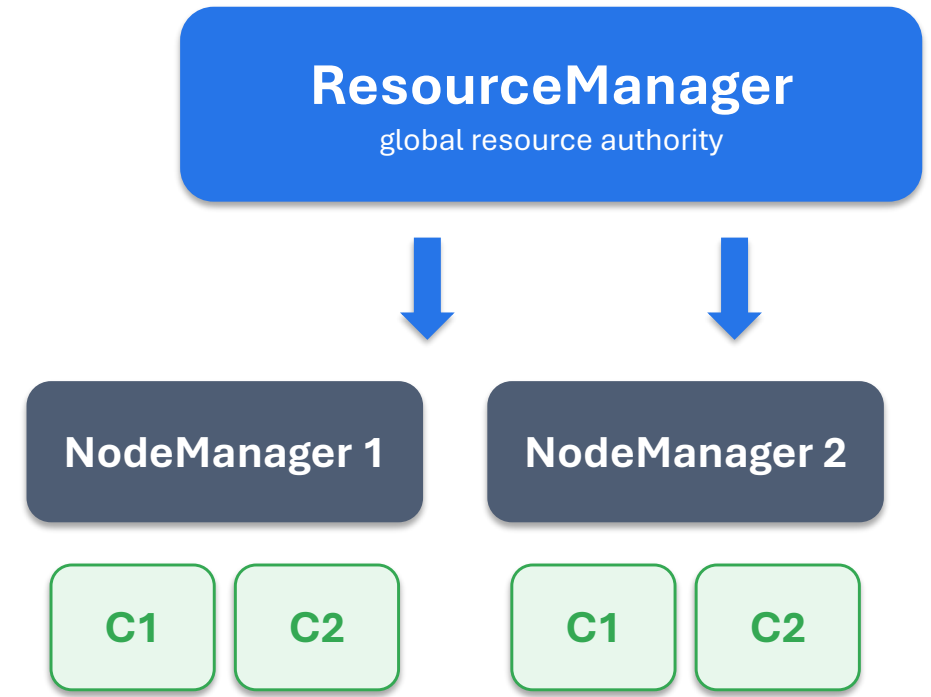


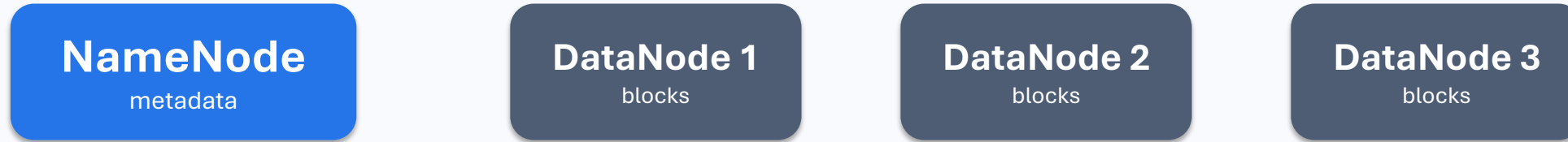
YARN

Resource Management for Hadoop

Who gets CPU and memory in a shared cluster?



We Already Know Where the Data Lives

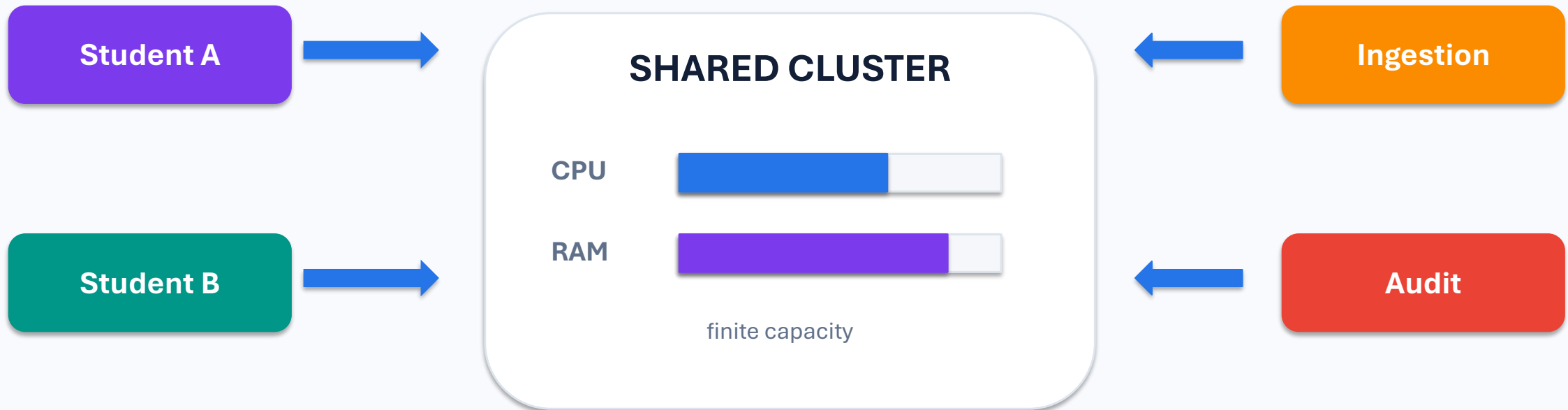


HDFS répond à la question : « Où sont les données ? »

Mais qui exécute le traitement ?

HDFS stocke les données ; YARN organise l'exécution des traitements.

The Shared Cluster Problem



Qui passe en premier ? Combien de mémoire
? Combien de CPU ?

Plusieurs utilisateurs et applications se disputent les mêmes ressources physiques.

Without a Resource Manager...

MONOPOLY

Un job consomme tout le cluster

OOM

La mémoire est dépassée

STARVATION

Des applications n'obtiennent rien

NO PRIORITY

Audit et test sont traités pareil

Sans arbitrage, un cluster partagé devient instable, injuste et difficile à exploiter.

YARN in One Sentence

YARN

Yet Another Resource Negotiator

Resources · Scheduling · Execution · Monitoring

YARN partage les ressources de calcul du cluster entre plusieurs applications.

Storage ≠ Compute

HDFS

Distributed storage

NameNode
DataNodes
Blocks & replication

YARN

Resource management

ResourceManager
NodeManagers
Applications & containers

HDFS et YARN sont complémentaires : l'un stocke, l'autre orchestre le calcul.

YARN Architecture

Four roles, one shared cluster



ResourceManager, NodeManagers, ApplicationMaster et containers ont des responsabilités distinctes.

The ResourceManager

ResourceManager

one per YARN cluster

Accept applications Reçoit les soumissions

Allocate resources Arbitre mémoire et vCores

Track nodes Connaît les NodeManagers

Enforce policies Applique queues et limites

Le ResourceManager est l'autorité globale qui arbitre les ressources du cluster.

Inside the ResourceManager

Scheduler

- Queues
- Capacities
- Resource allocation
- Scheduling policy

ApplicationsManager

- Accept submissions
- Launch first AM container
- Restart AM if needed

Le Scheduler alloue les ressources ; l'ApplicationsManager gère l'admission et le premier container.

The NodeManager

NodeManager
one per worker node



Launch containers



Monitor resources



Collect logs



Send heartbeats

Chaque NodeManager exécute localement les containers et remonte l'état de son nœud.

The ApplicationMaster

ApplicationMaster

one per application

1

**Request
containers**

ResourceManager

2

Launch tasks

NodeManagers

3

Track progress

Application tasks

4

Handle failures

Retry / report

Chaque application possède son propre ApplicationMaster pour négocier et suivre son exécution.

What Is a YARN Container?

YARN CONTAINER

4 GB RAM

2 vCores

on NodeManager 2

Resource allocation

Execution environment

Tracked by the NodeManager

Un container YARN est une allocation de ressources sur un nœud : mémoire, vCores et contexte d'exécution.

YARN Container ≠ Docker Container

YARN CONTAINER

Allocation de ressources
Mémoire + vCores
Pilotée par YARN

DOCKER CONTAINER

Isolation de processus
Image + filesystem
Pilotée par Docker

Le même mot “container” désigne deux concepts différents.

Node ≠ Application ≠ Container

NODE

Machine du cluster

NodeManager
32 GB RAM
8 vCores

APPLICATION

Travail soumis

WordCount
Spark job
Hive query

CONTAINER

Allocation locale

4 GB RAM
2 vCores
Task process

Le nœud fournit des ressources, l'application porte un travail, le container exécute une partie de ce travail.

CPU + Memory Are Resources

Container request

MEMORY

4096 MB

vCORES

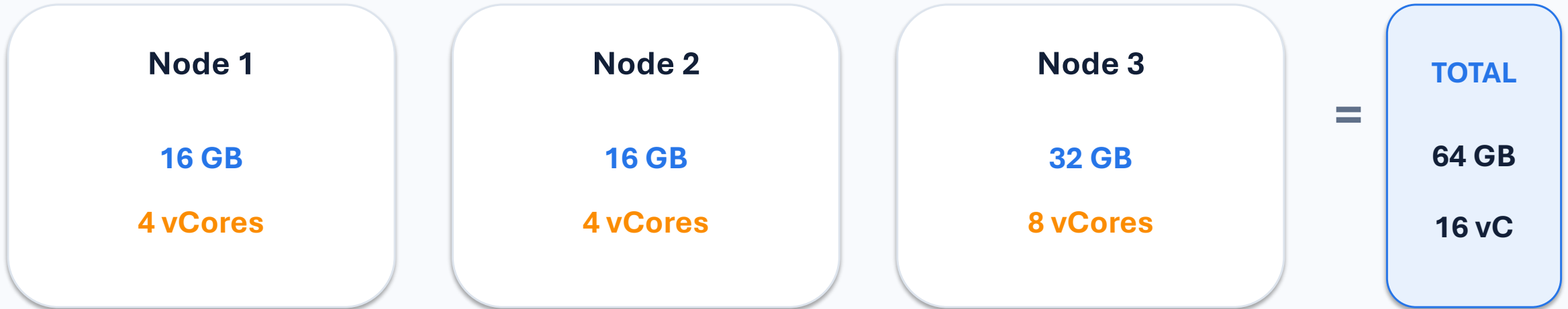
2

Scheduler decision

Is there a node with
enough available resources
and a policy allowing the allocation?

Le scheduler raisonne sur des demandes et des capacités de ressources.

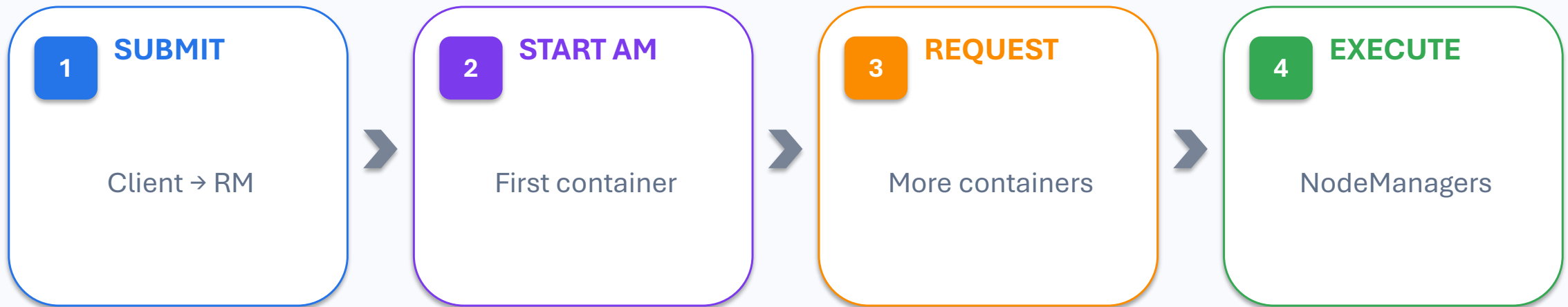
Cluster Capacity



Capacité totale $\neq$ capacité toujours libre : certains containers sont déjà en cours.

La capacité YARN agrège les ressources disponibles sur les NodeManagers.

How an Application Starts



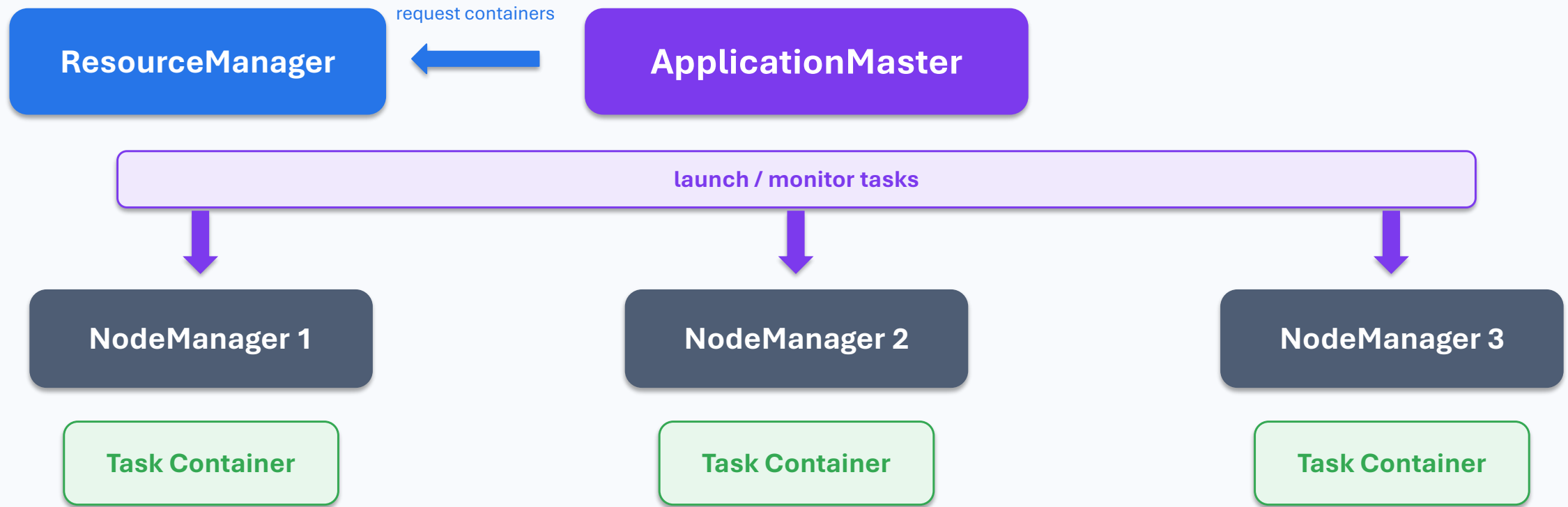
Une application YARN progresse de la soumission à l'exécution par étapes distinctes.

1-2 · Submit & Launch the ApplicationMaster



Le ResourceManager accepte la soumission puis démarre l'ApplicationMaster dans un premier container.

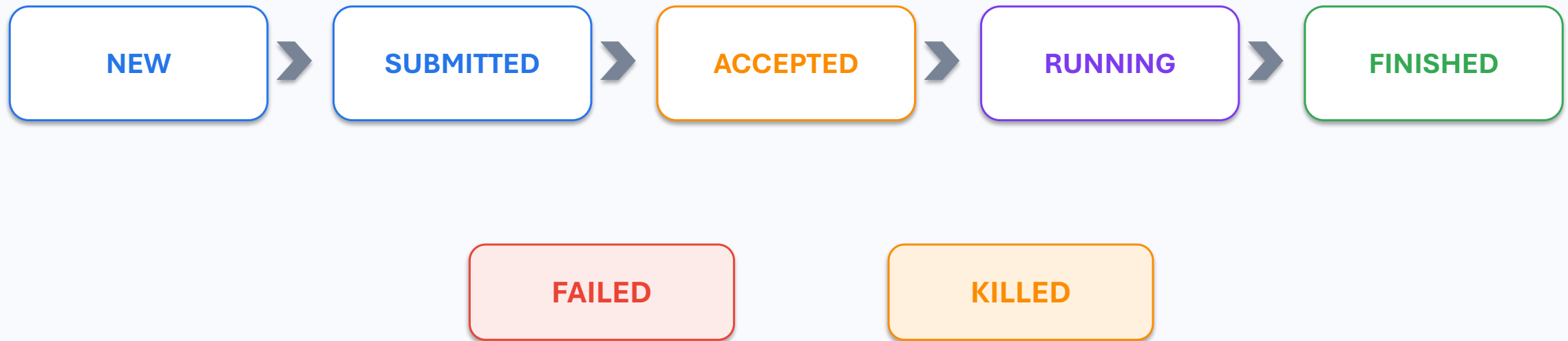
3-4 · Request Containers & Execute Tasks



Tasks run in parallel across the cluster

L'ApplicationMaster négocie les containers puis les NodeManagers exécutent les tâches.

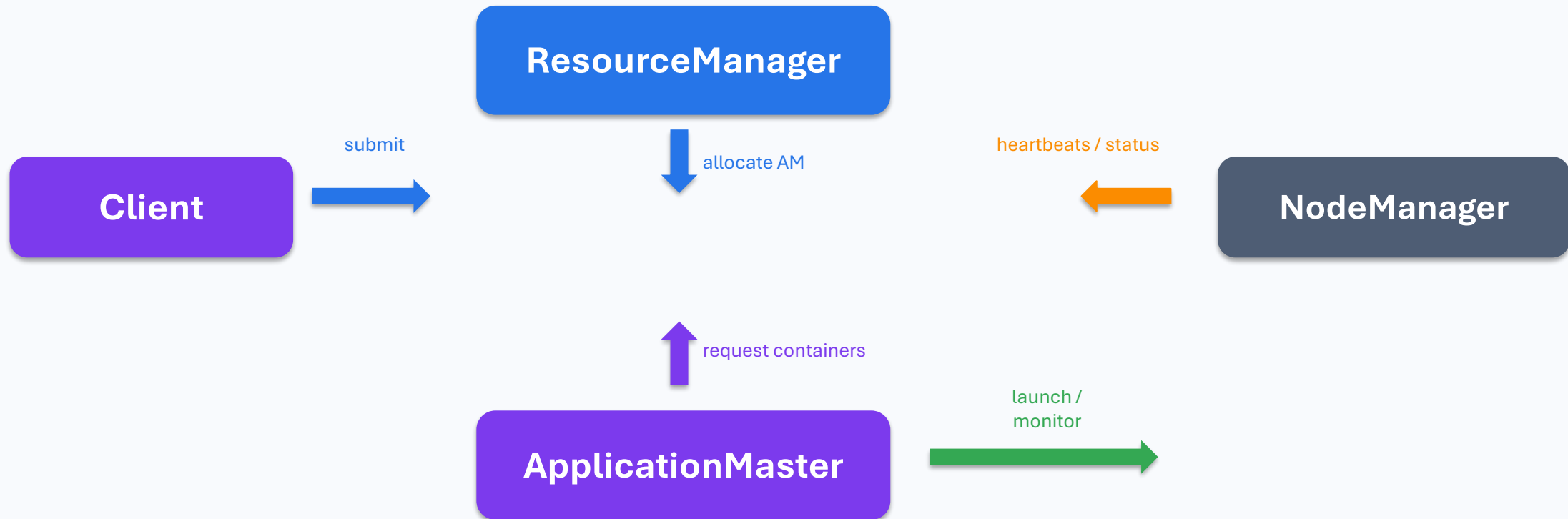
Application Lifecycle



Final status and diagnostics are available through YARN and the HistoryServer.

Les états YARN permettent de suivre une application depuis sa soumission jusqu'à son résultat final.

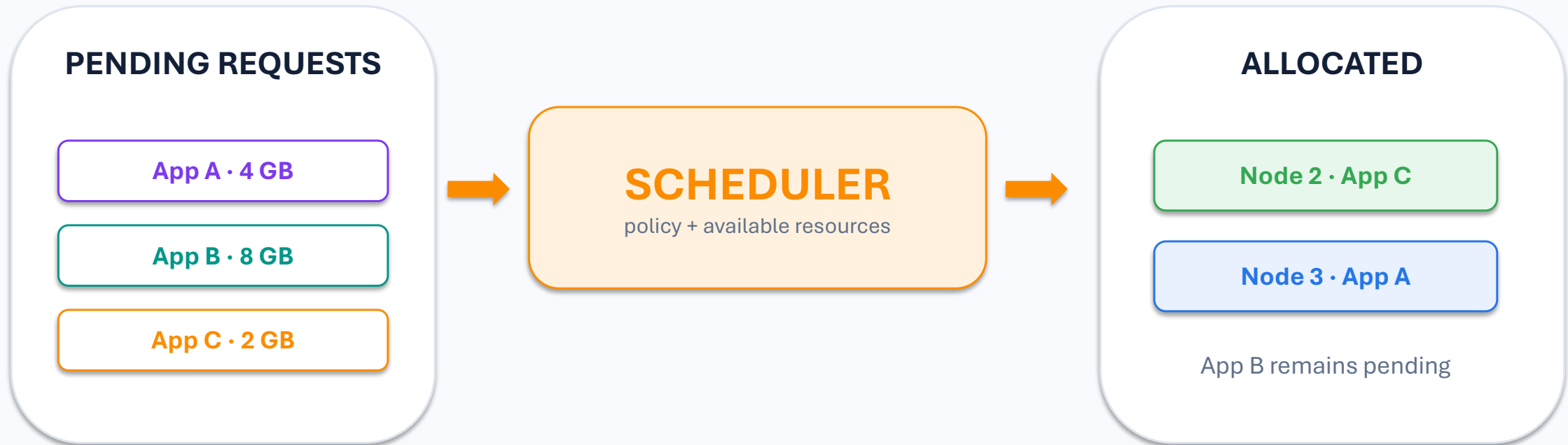
Who Talks to Whom?



Blue = admission/allocation · Purple = application negotiation · Green = task execution · Orange = node status

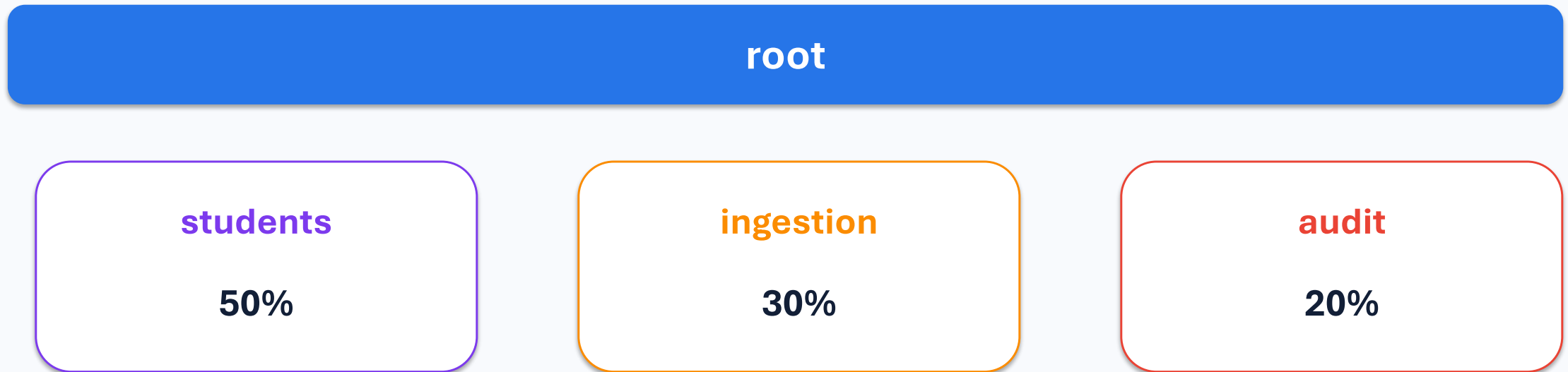
Chaque interaction répond à une responsabilité précise : soumettre, allouer, lancer ou surveiller.

Scheduling



Le scheduler choisit quelles demandes de containers peuvent être satisfaites, où et quand.

Why Queues?



Queues = capacity guarantees + limits + access control + visibility

Les queues séparent les usages, appliquent des garanties et limitent les effets d'un utilisateur ou d'un job.

CapacityScheduler

root = 100%



Unused audit capacity



may be used temporarily by another queue
when policy and maximum capacity allow it

CapacityScheduler permet à plusieurs tenants de partager un cluster avec garanties et élasticité.

Capacity vs Maximum vs Current Usage

**GUARANTEED
CAPACITY**

30%

Minimum share when demand exists

**MAXIMUM
CAPACITY**

70%

Upper bound for elasticity

**CURRENT
USAGE**

45%

What the queue consumes now

Une queue possède une garantie, une limite éventuelle et une consommation qui varie dans le temps.

Multi-Tenancy & Governance

User limits

Éviter qu'un utilisateur monopolise une queue

Queue limits

Protéger les autres usages

ACLs

Contrôler qui peut soumettre

Priorities

Favoriser certains traitements

Le scheduling est un sujet de gouvernance autant qu'un mécanisme technique.

A DORA Scheduling Scenario

INGESTION

Keep receiving logs

HIGH**AUDIT / FORENSIC**

Investigate an incident

HIGH**DAILY ARCHIVE**

Long-term retention

NORMAL**EXPLORATION**

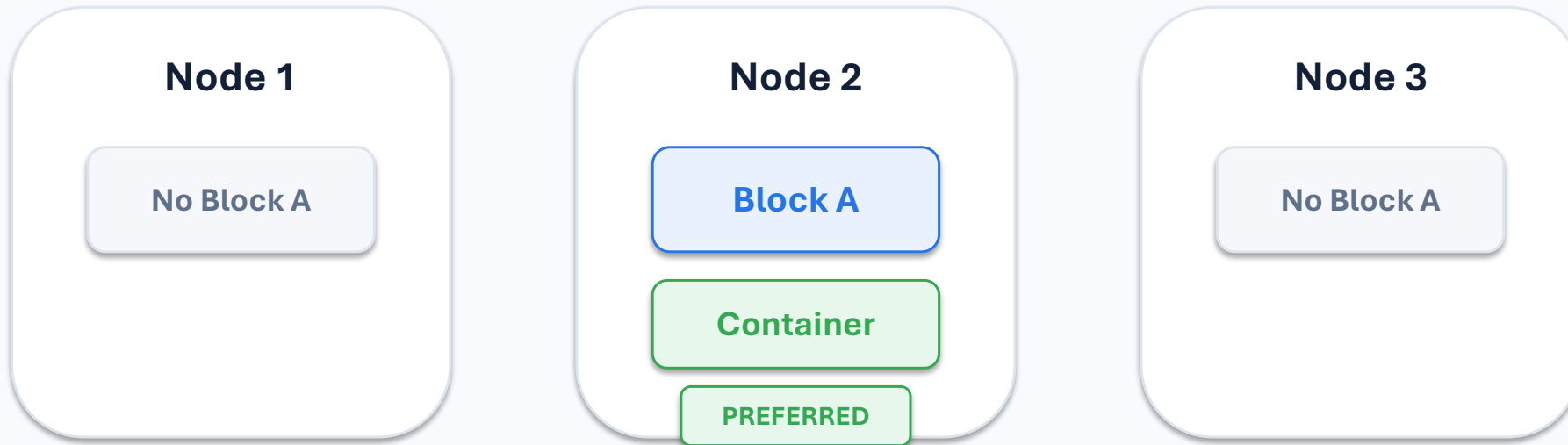
Ad-hoc student analysis

LOW

Question : quelles queues et quelles limites garantissent cette politique ?

En cas d'incident, les traitements critiques doivent obtenir les ressources au bon moment.

Data Locality



Data-local > rack-local > off-rack, selon la disponibilité et la politique.

Lorsque c'est possible, YARN rapproche le calcul des données afin de réduire le trafic réseau.

Monitoring & Diagnostics

ResourceManager UI

Nodes · apps · queues · resources

YARN CLI

status · list · queue status

Application logs

stdout · stderr · diagnostics

HistoryServer

MapReduce jobs after completion

Diagnostic path: state → diagnostics → container logs → input/output → resources

ResourceManager UI, CLI, logs et HistoryServer offrent des vues complémentaires.

MapReduce as a YARN Application



Observe in TP 03

application ID · containers · duration · logs · output

Aujourd'hui : observer comment YARN exécute WordCount. Prochaine séance : comprendre Mapper, Shuffle et Reducer.

Key Takeaways

1

ResourceManager

global
arbitration

2

NodeManager

local
execution

3

ApplicationMaster

per-app
coordination

4

Container

resource
allocation

5

Queues

multi-tenant
governance

TP 03: yarn node • yarn application • queue status • WordCount • logs

YARN transforme un ensemble de machines en plateforme de calcul partagée et gouvernée.